

NAVMARKET OPS · TRAINING COURSE

The Foundations of Computing: From Hardware to the Cloud

Hardware, operating systems, networking, virtualization, cloud, containers and Kubernetes: the layers every DevOps and cloud engineer works on, explained from the ground up.

October 2026

· @Navatech

Contents

1. The computer and its hardware	2	8. Containers and Docker	19
2. Software and the operating system	3	9. Kubernetes	22
3. Networking basics	5	10. Virtualization vs Docker vs	
4. TCP/IP in detail	7	Kubernetes vs cloud	25
5. Virtualization	11	11. Reliable sources to go further	27
6. From virtualization to the cloud	13	12. Self-assessment	29
7. The cloud	15		

How to use this course

Everything you do in DevOps and cloud engineering is a stack of layers: each layer hides the complexity of the one below it. Understanding the stack means understanding why each tool exists.

1. **Hardware:** chips that compute, store and transmit bits.
2. **Operating system:** the software that shares this hardware between programs.
3. **Networking:** how two machines talk to each other (TCP/IP).
4. **Virtualization:** several "logical" machines on a single physical machine.
5. **Cloud:** virtualization rented on demand, driven by APIs, at global scale.
6. **Containers (Docker):** isolating an application rather than an entire machine.
7. **Orchestration (Kubernetes):** running hundreds of containers across several machines.

Read the course in order once (2 to 3 hours), then come back to the section that matches the current session of the NavMarket programme: S1-2 → sections 1-2, S3-4 → sections 3-4 (networking), S7 → section 5 (virtualization), S11 → section 8 (Docker), S13 → sections 6-7 (cloud), S19 → section 9 (Kubernetes). Each section points to the official sources listed at the end.

1. The computer and its hardware

A computer does three things: it **computes** (CPU), it **remembers** (RAM and storage) and it **communicates** (network, peripherals). A server is the same thing, optimized to run 24/7 without a screen.

Component	Role	What matters in ops
CPU (processor)	Executes instructions. Several cores = several tasks in parallel.	Number of cores/threads, Intel VT-x / AMD-V virtualization extensions (required for KVM). In the cloud, we talk about vCPUs .
RAM (main memory)	Holds the data currently in use. Very fast, wiped at shutdown.	Not enough RAM = the machine starts "swapping" or the kernel kills processes (OOM killer).
Storage (HDD, SSD, NVMe)	Keeps data persistently.	Capacity, throughput, IOPS (operations per second). Databases are very sensitive to it.
Motherboard and bus (PCIe)	Connects all the components.	Determines what can be plugged in, and at what speed.
Network card (NIC)	Sends and receives frames over a cable or Wi-Fi.	Carries a unique MAC address ; 1, 10, 25 or 100 Gbit/s in a datacenter.
GPU	Massively parallel computing.	Display, but above all AI and scientific computing.
BIOS / UEFI firmware	The first program to run at boot.	Enables VT-x and manages Secure Boot (something you run into when setting up a dual boot).

How a machine boots, in order: power on → UEFI tests the hardware → UEFI starts the bootloader (GRUB on Ubuntu) → GRUB loads the Linux kernel into RAM → the kernel starts the first process (systemd, PID 1) → systemd starts the services (SSH, Nginx...).

Bits and units: 1 byte = 8 bits. Storage is measured in bytes (GB, TB), network throughput in bits per second (Mbit/s). A 1 Gbit/s fiber connection therefore transfers at most ~125 MB/s.

Server vs PC: a server has ECC RAM (which corrects errors), redundant power supplies and disks (RAID), a remote management card (iDRAC, iLO), and it is mounted in a datacenter rack. These are the machines the cloud runs on.

2. Software and the operating system

The operating system (OS) is the referee between the hardware and the programs: it decides who uses the CPU, the RAM, the disk and the network, and it stops programs from stepping on each other.

Families of software

- **Firmware:** embedded close to the hardware (UEFI, a disk's firmware).
- **System:** kernel, drivers, core services.
- **Applications:** Nginx, PostgreSQL, a web browser, a NestJS app.
- **Libraries and runtimes:** shared code (glibc, OpenSSL) or execution environments (Node.js, Python, JVM).

Kernel space and user space. The kernel has full rights over the hardware. Programs run in user space and must go through **system calls** (syscalls: `open`, `read`, `fork`, `socket` ...) to ask the kernel for a service. Keep this boundary in mind: it is exactly what separates a VM from a container (section 8).

The 5 jobs of the OS

1. **Processes:** a running program, with a PID, an owner and memory. The scheduler shares the CPU between them thousands of times per second. Commands: `ps`, `top`, `htop`, `kill`.
2. **Memory:** each process believes it has its own memory (virtual memory); the kernel maps it to real RAM and spills over to disk (swap) when needed. Command: `free -h`.
3. **Files:** the filesystem (ext4, XFS, NTFS) organizes the disk into directories and files with permissions (`rw`, owner, group). On Linux, "everything is a file", even devices (`/dev/sda`). Commands: `ls -l`, `chmod`, `chown`, `df -h`.
4. **Devices:** drivers translate the kernel's orders for each piece of hardware.
5. **Networking and security:** the TCP/IP stack lives in the kernel; users, groups and `sudo` control who can do what.

Linux vs Windows

Criterion	Linux (Ubuntu, Debian, RHEL)	Windows / Windows Server
License	Free and open source (except RHEL support)	Paid
Server usage	The vast majority of cloud and web servers	Active Directory, legacy .NET applications
Administration	Command line (Bash), text files in <code>/etc</code>	Graphical interface + PowerShell
Containers	Native (Docker was born on Linux)	Through WSL2 or Windows containers

Linux directories to know: `/etc` (configuration), `/var/log` (logs), `/home` (users), `/usr/bin` (programs), `/tmp` (temporary files), `/proc` (live kernel information).

The **shell** (Bash) is a program that reads your commands and has the kernel execute them. A **service** (or daemon) is a process running in the background, managed by systemd: `systemctl status nginx`, `journalctl -u nginx`.

3. Networking basics

A network is a set of machines exchanging messages according to shared rules called **protocols**. To keep things manageable, the problem is split into **layers**: each layer provides a service to the layer above and ignores the details of the layer below.

The OSI model (7 layers, theoretical) and the TCP/IP model (4 layers, the one the Internet uses)

OSI	TCP/IP	What it does	Unit	Address	Examples	Device
7 Application, 6 Presentation, 5 Session	Application	The service delivered to the user, encryption, format	Data	Name (DNS), URL	HTTP(S), DNS, SSH, TLS, SMTP	Server, proxy, L7 load balancer
4 Transport	Transport	Delivers data to the right program, reliably or not	Segment (TCP) / datagram (UDP)	Port (80, 443, 22)	TCP, UDP	Firewall, L4 load balancer
3 Network	Internet	Carries data from one network to another, across the world	Packet	IP address	IPv4, IPv6, ICMP (ping)	Router
2 Data link	Network access	Delivers to a direct neighbor on the same local network	Frame	MAC address	Ethernet, Wi-Fi, ARP, VLAN	Switch
1 Physical	Network access	Carries the bits	Bit	—	Copper cable, fiber, radio waves	Cable, repeater

OSI mnemonic, bottom to top: Please Do Not Throw Sausage Pizza Away (Physical, Data link, Network, Transport, Session, Presentation, Application).

Encapsulation: when your browser sends an HTTP request, each layer wraps the data in its own header, like nested envelopes. HTTP → + TCP header (ports) → + IP header (IP addresses) → + Ethernet header (MAC addresses). The recipient opens the envelopes in reverse order. Wireshark shows you exactly these envelopes.

The three addresses never to confuse

- **MAC** (`52:54:00:ab:cd:ef`): identifies a network card and is only used on the local network. It never crosses a router.
- **IP** (`192.168.1.20`): identifies a machine on a network and makes it reachable from anywhere. It stays the same end to end (except with NAT).
- **Port** (`443`): identifies the program on the machine. `IP + port = socket` (`192.168.1.20:443`).

Analogy: the IP is the building's address, the port is the apartment number, and the MAC is the name of the mail carrier covering the last leg, which changes at every leg.

Switch vs router

- A **switch** connects the machines of the same local network (LAN). It learns which MAC address sits behind which port and forwards frames there.
- A **router** connects different networks together. It reads the destination IP and picks the next hop using its **routing table**. A home Internet box is at once a router, a switch, a Wi-Fi access point, a DHCP server and a NAT device.

Topologies and sizes: LAN (home, office), WAN (links distant sites), Internet (the network of networks). A **VLAN** splits a single switch into several isolated logical networks.

4. TCP/IP in detail

TCP/IP is the Internet's protocol suite: IP finds the path to the right machine, TCP or UDP delivers the data to the right program. Everything else (DNS, DHCP, HTTP, SSH) is built on top of it.

4.1 The IPv4 address

An IPv4 address is 32 bits long, written as 4 decimal bytes: `192.168.1.20`. It has two parts: the **network** part (the neighborhood) and the **host** part (the house). The **netmask** says where the split is.

- `192.168.1.20/24` = netmask `255.255.255.0`: the first 24 bits (`192.168.1`) are the network, the last byte is the host.
- In every network, the first address is the **network address** (`192.168.1.0`) and the last one is the **broadcast address** (`192.168.1.255`): neither can be assigned.
- Number of hosts = $2^{(32 - \text{prefix})} - 2$.

CIDR notation	Netmask	Addresses	Usable hosts	Typical use
/8	255.0.0.0	16,777,216	16,777,214	Large private network <code>10.0.0.0/8</code>
/16	255.255.0.0	65,536	65,534	A whole AWS VPC (<code>10.0.0.0/16</code>)
/24	255.255.255.0	256	254	A LAN, an AWS subnet
/26	255.255.255.192	64	62	Small subnet
/28	255.255.255.240	16	14	The smallest subnet AWS allows
/30	255.255.255.252	4	2	Point-to-point link between 2 routers
/32	255.255.255.255	1	1 (a single machine)	Firewall rule for one specific IP

Private addresses (RFC 1918, not routable on the Internet): `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`. Special addresses: `127.0.0.1` (localhost, the machine itself), `0.0.0.0` (all interfaces, or the "default route" as `0.0.0.0/0`), `169.254.x.x` (no DHCP server found).

Subnetting: taking `10.0.0.0/16` and cutting it into /24s gives 256 subnets (`10.0.0.0/24` , `10.0.1.0/24` ...). This is exactly what you do on AWS: one public and one private subnet per zone. Practice until you can do it in your head.

4.2 IPv6

IPv4 only offers ~4.3 billion addresses, and they have run out. IPv6 uses 128 bits, written in hexadecimal: `2001:db8::1`. No more need for NAT, and automatic configuration (SLAAC). Both coexist today ("dual stack").

4.3 TCP vs UDP

	TCP	UDP
Connection	Yes, set up before sending (handshake)	No, data is sent right away
Reliability	Acknowledgments, retransmission, guaranteed order	No guarantee
Flow and congestion control	Yes	No
Speed	Slower, more header overhead	Very fast, lightweight
Examples	HTTP/1.1 and 2, SSH, PostgreSQL, SMTP	DNS, DHCP, VoIP, games, streaming, HTTP/3 (QUIC)

The TCP three-way handshake (opening a connection):

1. The client sends **SYN** ("I want to talk").
2. The server replies **SYN-ACK** ("OK, me too").
3. The client sends **ACK** ("here we go"). The connection is established.

Closing uses FIN / ACK on each side; an abrupt cut sends RST. A closed port replies with **RST**; a firewall that blocks traffic replies with nothing (**timeout**). This difference is a great troubleshooting clue.

4.4 The protocols that make the network work

- **ARP:** "who has IP 192.168.1.1? Give me your MAC." Maps IP → MAC on the local network. Command: `ip neigh`.
- **DHCP:** automatically gives a newly connected machine its IP, netmask, gateway and DNS server. 4 steps: Discover, Offer, Request, Ack ("DORA").

- **DNS:** translates a name (`navmarket.lan`, `google.com`) into an IP address. The resolver queries the root servers, then the `.com` servers, then the authoritative server for the domain. Key records: `A` (IPv4), `AAAA` (IPv6), `CNAME` (alias), `MX` (mail), `TXT`, `NS`. Tools: `dig`, `nslookup`, the `/etc/hosts` file.
- **Default gateway:** the router to which a machine sends everything that is not on its local network. Command: `ip route`.
- **Routing:** each router looks at the destination IP, finds the most specific route in its table and forwards the packet to the next router (a "hop"). Command: `traceroute`.
- **NAT:** on the way out, the Internet box replaces the machine's private IP with its own public IP and remembers the mapping so it can send the reply back. **Port forwarding** (or DNAT) does the opposite: it exposes an internal service to the Internet.
- **ICMP:** control and error messages. `ping` uses it.
- **TLS:** encrypts a TCP connection (HTTPS = HTTP + TLS). The server proves its identity with a **certificate** (Let's Encrypt, lab S4.6).

4.5 Ports to know by heart

Port	Protocol	Service
22	TCP	SSH (and SFTP)
25 / 587	TCP	SMTP (sending email)
53	UDP and TCP	DNS
67 / 68	UDP	DHCP server / client
80	TCP	HTTP
123	UDP	NTP (time)
443	TCP (and UDP for HTTP/3)	HTTPS
3000	TCP	Next.js / Node in development (convention)
3306	TCP	MySQL
5432	TCP	PostgreSQL
6379	TCP	Redis
6443	TCP	Kubernetes API
8080	TCP	Alternative HTTP, proxies, Java applications

Ports 0-1023 are the "well-known" ports, which only root can open. The client, for its part, uses a random **ephemeral port** (32768-60999 on Linux).

4.6 What happens when you type `https://navmarket.lan`

1. The browser asks DNS for the IP of `navmarket.lan` (cache, `/etc/hosts`, then the resolver over UDP 53).
2. If the server is on the same network, ARP finds its MAC; otherwise ARP finds the gateway's MAC.
3. TCP opens a connection to that IP on port 443 (SYN, SYN-ACK, ACK).
4. TLS negotiates encryption and checks the certificate.
5. The browser sends the HTTP request `GET /`; Nginx replies `200 OK` with the HTML.
6. Every packet goes through the layers and the routers, encapsulated and decapsulated at each step.

This is a classic interview question: be able to walk through it in 2 minutes.

4.7 Linux troubleshooting commands

`ip a` (interfaces and IPs) · `ip route` (routes) · `ping` (reachable?) · `traceroute` / `mtr` (path) · `dig` (DNS) · `ss -tulpn` (listening ports) · `curl -v` (detailed HTTP request) · `nc -zv host port` (is the port open?) · `tcpdump` / Wireshark (see the packets).

5. Virtualization

Virtualizing means making several operating systems believe they each have their own machine, while they actually share a single physical server. The software that performs this trick is called the **hypervisor**.

5.1 Why virtualize

Before (1990s-2000s), the rule was "one application = one physical server", to avoid conflicts. The result: servers used at 10-15% of their capacity, full server rooms, and weeks to order a new one. Virtualization fixes this:

- **Consolidation:** 10 to 20 VMs on one server, so less hardware, electricity and space.
- **Isolation:** a VM that crashes or gets hacked does not affect the others.
- **Speed:** creating a VM takes a few minutes, not weeks.
- **Mobility:** a VM is a set of files; it can be copied, backed up, or moved live to another server (**live migration**).
- **Snapshots:** capture the state of a VM before a risky operation and roll back to it. Ideal for labs.

5.2 The two types of hypervisors

	Type 1 "bare metal"	Type 2 "hosted"
Where it runs	Directly on the hardware; it is the OS	Like an application, on top of a regular OS
Performance	Maximum	Lower (one extra layer)
Use	Datacenters, cloud, production	Workstations, testing, learning
Examples	VMware ESXi, Microsoft Hyper-V, Xen, KVM (built into the Linux kernel), Proxmox VE (based on KVM)	VirtualBox, VMware Workstation, Parallels

The case of KVM: it is a Linux kernel module that turns Linux itself into a type 1 hypervisor. QEMU emulates the devices, libvirt provides a common API and virt-manager provides the graphical interface. AWS EC2 now runs on **Nitro**, a hypervisor derived from KVM.

5.3 How it works

- **CPU:** thanks to Intel VT-x / AMD-V, the processor can run the VM's instructions directly and hands control back to the hypervisor for sensitive operations. Each VM sees **vCPUs**.
- **Memory:** the hypervisor gives each VM a range of RAM that it believes is physical (EPT / NPT extensions for fast translation). Memory can be **overcommitted** (giving out more RAM in total than actually exists), with care.
- **Disk:** a VM's disk is a file (`.qcow2` for KVM, `.vmdk` for VMware, `.vhdx` for Hyper-V), thin-provisioned (grows as needed) or fixed-size.
- **Devices:** emulated (slow) or paravirtualized (`virtio` drivers, fast, because the guest OS knows it is virtualized).
- **Vocabulary:** the physical machine = the **host**; the VM = the **guest**.

5.4 Virtual networking

The hypervisor creates virtual switches (Linux bridges, Open vSwitch). Three modes to know:

- **NAT:** the VM reaches the Internet by borrowing the host's IP, but cannot be reached from outside. This is virt-manager's default mode (`default` network, `192.168.122.0/24`).
- **Bridge:** the VM is plugged into your real network and gets its own IP there, like one more PC.
- **Isolated / host-only network:** the VMs talk to each other and to the host, without Internet access. Ideal for simulating a corporate network.

5.5 Beyond VMs

Storage is virtualized too (SAN, Ceph: disks from several machines seen as a single one), and so is the network (SDN: software-defined networking, VLAN, VXLAN). When compute, storage and networking are all virtualized and software-driven, you get a **software-defined datacenter**: the direct foundation of the cloud.

Market players: VMware vSphere (the historical leader, acquired by Broadcom in 2023, which pushed many companies toward Proxmox, Hyper-V or Nutanix), Microsoft Hyper-V, Proxmox VE, Nutanix AHV, KVM/OpenStack, Citrix XenServer.

6. From virtualization to the cloud

The cloud is not a new technology: it is virtualization plus **automation**, **self-service** and **pay-as-you-go billing**, at the scale of giant datacenters. Virtualization turned servers into software; the cloud made those servers available through a simple API call.

6.1 What virtualization alone did not solve

With VMware in its own datacenter, a company still had to:

- buy and renew the hardware every 3 to 5 years, and size it for the peak (Black Friday) even if it sat idle the rest of the year;
- manage the building, power, cooling, network and physical security;
- file a ticket with the infrastructure team to get a VM (often taking days);
- pay upfront (**CAPEX**, capital expenditure) instead of paying for what it actually uses.

6.2 The three ingredients the cloud added

1. **Self-service through APIs**: everything is created in seconds, from a web console, the command line or code (`aws ec2 run-instances`, Terraform). This is what makes Infrastructure as Code and DevOps possible.
2. **Elasticity**: add 50 servers for 2 hours, then give them back. Auto Scaling does it automatically based on load.
3. **Pay-as-you-go (OPEX, operating expenditure)**: by the second or by the hour, with no commitment.

On top of this come **scale** (millions of servers shared between customers, hence cheaper), **global reach** (regions on every continent) and **managed services**: the provider runs the database, the backups and the updates for you.

6.3 Key dates

Year	Event	Why it matters
1972	IBM VM/370	First commercial virtualization, on mainframes
1999-2001	VMware Workstation, then ESX	Virtualization reaches ordinary x86 servers
2003	Xen (open source)	The hypervisor AWS EC2 ran on in its early days
2005-2006	Intel VT-x and AMD-V	The processor helps the hypervisor: fast, simple virtualization
2006	AWS launches S3, then EC2	Birth of the modern public cloud: VMs rented by the hour through an API
2007	KVM merged into the Linux kernel	Linux becomes a hypervisor, for free
2008-2010	Google App Engine, Microsoft Azure, OpenStack	Competition, and open-source private cloud
2011	NIST definition of cloud computing (SP 800-145)	Official vocabulary: IaaS, PaaS, SaaS
2013	Docker	Easy-to-use containers: you ship the application, not the VM
2014	Kubernetes (Google), AWS Lambda	Container orchestration and serverless
2017	AWS Nitro	In-house hypervisor derived from KVM, dedicated hardware
2020s	Cloud native, multi-cloud, AI	Kubernetes becomes the standard; the cloud hosts most new projects

6.4 The thread to remember

Dedicated physical server → VM (the machine is shared) → IaaS cloud (VMs rented on demand) → PaaS and managed services (no more OS to manage) → containers (you ship just the application and its dependencies) → Kubernetes (thousands of containers orchestrated) → serverless (you ship just a function). At each step you hand over one more layer and gain speed, at the cost of less control.

7. The cloud

The cloud means renting, on demand, over the Internet and through APIs, computing power, storage, networking and ready-made services, and paying for what you use. The "cloud" is in fact very physical: hundreds of datacenters full of servers.

7.1 The official definition (NIST): 5 characteristics

1. **On-demand self-service:** you create your resources yourself, without talking to anyone.
2. **Broad network access:** reachable from anywhere, over the Internet.
3. **Resource pooling:** the hardware is shared between many customers (multi-tenant), isolated from one another.
4. **Rapid elasticity:** capacity scales up or down in minutes.
5. **Measured service:** everything is metered and billed by usage.

7.2 Service models: who manages what

Layer	On-premises	IaaS	PaaS	SaaS
Application and data	You	You	You	Provider (you manage your data and your access)
Runtime, middleware	You	You	Provider	Provider
Operating system	You	You	Provider	Provider
Virtualization	You	Provider	Provider	Provider
Servers, storage, network	You	Provider	Provider	Provider
Datacenter, power	You	Provider	Provider	Provider
Examples	A server in your own office	AWS EC2, Azure VMs, Oracle Cloud	AWS Elastic Beanstalk, Heroku, Vercel, Google App Engine	Gmail, Microsoft 365, Salesforce, GitHub

The pizza analogy: homemade (on-premises), take-and-bake kit (IaaS), delivered (PaaS), eating out at a restaurant (SaaS). In between sit **CaaS** (managed containers: ECS, EKS) and **FaaS / serverless** (you ship a function: AWS Lambda).

7.3 Deployment models

- **Public cloud:** a provider's shared infrastructure (AWS, Azure, Google Cloud, OVHcloud, Scaleway).
- **Private cloud:** dedicated to a single organization, on its premises or hosted (OpenStack, VMware, Proxmox with automation).
- **Hybrid:** a connected mix of both (sensitive data in-house, the rest in the cloud).
- **Multi-cloud:** several public providers at the same time.
- **Sovereign cloud:** hosted and operated under European jurisdiction (the ANSSI's SecNumCloud label in France).

7.4 The global infrastructure (AWS example)

- **Region:** a geographic area (`eu-west-3` = Paris). Your data stays there unless you move it.
- **Availability Zone (AZ):** one or more isolated datacenters within a region (`eu-west-3a`, `b`, `c`). Running your servers in at least 2 AZs = surviving the failure of a datacenter.
- **Points of presence (edge):** servers close to users, used for caching (CloudFront, CDN).

7.5 The shared responsibility model

AWS is responsible for security **of** the cloud (buildings, hardware, hypervisor, global network). You are responsible for security **in** the cloud: your data, IAM access, firewall rules, OS updates on your VMs, encryption. Most data leaks come from customer mistakes (a public S3 bucket, an access key pushed to GitHub).

7.6 Core services and their equivalents

Need	AWS	Azure	Google Cloud	What it replaces
Virtual machines	EC2	Virtual Machines	Compute Engine	Your KVM VMs
Private network	VPC (subnets, route tables, Internet gateway, NAT Gateway)	Virtual Network	VPC	Your LAN and Internet box
Firewall	Security Groups, Network ACLs	Network Security Groups	Firewall rules	<code>ufw</code> , <code>iptables</code>
Object storage	S3	Blob Storage	Cloud Storage	An unlimited file share
VM disks	EBS	Managed Disks	Persistent Disk	The <code>.qcow2</code> file
Managed database	RDS, Aurora	Azure SQL, Database for PostgreSQL	Cloud SQL	A PostgreSQL installed by hand
Identities and permissions	IAM	Entra ID	Cloud IAM	Linux users and <code>sudo</code>
DNS	Route 53	Azure DNS	Cloud DNS	Your own DNS server (<code>navmarket.lan</code>)
Load balancing	ELB (ALB, NLB)	Load Balancer, Application Gateway	Cloud Load Balancing	Nginx as a reverse proxy
Containers	ECS, Fargate, EKS	Container Apps, AKS	Cloud Run, GKE	Docker and Kubernetes on your own machines
Serverless	Lambda	Functions	Cloud Run functions	A script running on a server
Monitoring	CloudWatch	Azure Monitor	Cloud Monitoring	Prometheus, Grafana, <code>journalctl</code>
Infrastructure as Code	CloudFormation (or Terraform)	Bicep (or Terraform)	Terraform	Clicking around in virt-manager

7.7 Benefits and pitfalls

- **Pros:** speed, no hardware to buy, elasticity, global reach, managed services, high availability within everyone's reach.

- **Cons:** the bill can explode if you forget resources (turn on budget alerts from day one), vendor lock-in, data egress costs, data sovereignty, complex IAM permissions. The discipline that manages cloud costs is called **FinOps**.

8. Containers and Docker

A container is an ordinary Linux process, enclosed in a bubble that makes it believe it is alone on the machine. Unlike a VM, it has no kernel of its own: **all containers share the host's kernel**. That is why a container starts in a second and weighs a few MB instead of several GB.

8.1 The problem it solves

"It works on my machine": the application runs on the developer's laptop but not in production, because of a different Node version, library or configuration. A container packages the application with all its dependencies into a single **image** that runs identically everywhere: a laptop, a VPS, AWS.

8.2 How Linux isolates a container

- **Namespaces**: what the process can **see**. It gets its own PIDs (it believes it is PID 1), its own network, mount points, hostname and users.
- **cgroups**: what the process can **consume**. Limits on CPU, RAM and disk (`docker run --memory 512m`).
- **Layered filesystem** (OverlayFS): the image is made of read-only layers, and the container adds a thin writable layer.

An important consequence: a Linux container needs a Linux kernel. On Windows or macOS, Docker Desktop actually starts a small Linux VM (WSL2 on Windows). And the isolation is weaker than a VM's, since the kernel is shared.

8.3 Docker vocabulary

Term	Definition	Analogy
Dockerfile	A text recipe describing how to build the image	The recipe
Image	An immutable package: a minimal filesystem + the app + its dependencies + the startup command	The frozen meal (or a class in programming)
Container	A running image; ephemeral and disposable	The reheated meal (or an object, an instance of the class)
Registry	A store of images: Docker Hub, GHCR (GitHub Container Registry), Amazon ECR	The supermarket
Volume	Storage that survives the deletion of the container	The external hard drive
Docker network	A virtual switch between containers, with DNS by service name	The containers' LAN
Docker Compose	A YAML file describing several containers together (front end, API, database)	The full menu

8.4 A concrete example

```
FROM node:22-alpine          # lightweight base image
WORKDIR /app
COPY package*.json ./
RUN npm ci                  # each instruction = one layer (cached)
COPY . .
RUN npm run build
EXPOSE 3000
CMD ["node", "dist/main.js"] # the container's main process
```

Life cycle: `docker build -t navmarket-api .` → `docker run -d -p 8080:3000 navmarket-api` (port 8080 on the host forwards to port 3000 in the container) → `docker push ghcr.io/...` → on the server, `docker pull` then `docker run`. Other commands: `docker ps`, `docker logs`, `docker exec -it <id> sh`, `docker compose up -d`.

8.5 Principles to follow

- One main process per container (the API on one side, PostgreSQL on the other).

- Immutable and disposable: you do not fix a container in place; you rebuild the image and replace the container.
- Data goes in volumes, never inside the container.
- Configuration through environment variables; secrets never in the image.
- Small, secure images: `alpine` or `slim` base, multi-stage builds, non-root user.

Docker is only one tool among others: the image format is standardized by the **OCI**, and other engines run it (containerd, CRI-O, Podman). Kubernetes uses containerd or CRI-O, not Docker itself, but Docker images run on it unchanged.

9. Kubernetes

Kubernetes (K8s) is a conductor: you describe the **desired state** ("3 copies of my API, reachable on port 80") and it does everything to make reality match it, continuously, across a set of machines. Docker runs containers on one machine; Kubernetes drives them across a **cluster**.

9.1 The problems it solves

With 3 containers on one server, Docker Compose is enough. With 300 containers on 20 servers, you need answers to: which machine should run each container? What happens if a container or a server dies? How do you add copies when load increases? How do you update without downtime? How do containers find each other? Kubernetes automates all these answers.

9.2 Cluster architecture

- **Control plane**, the brain:
 - **kube-apiserver**: the front door; everything goes through its API (port 6443), and `kubectl` talks to it.
 - **etcd**: the key-value database that stores the cluster's state.
 - **kube-scheduler**: decides which node each pod runs on.
 - **controller-manager**: the loops that compare the desired state with the actual state and correct the difference.
- **Worker nodes**, the muscles (VMs or physical machines):
 - **kubelet**: the agent that starts and monitors the pods on the node.
 - **Container runtime** (containerd): actually runs the containers.
 - **kube-proxy**: manages the network rules for Services.

9.3 Essential objects

Object	Role
Pod	The smallest unit: one or more containers sharing the same IP and volumes. Mortal and replaceable.
Deployment	Describes how many copies (replicas) of a pod to run and how to update them (rolling update, rollback).
ReplicaSet	Managed by the Deployment; keeps the right number of pods running.
Service	A stable address (IP + DNS name) in front of a changing group of pods, with load balancing. Types: ClusterIP (internal), NodePort, LoadBalancer.
Ingress / Gateway API	HTTP(S) entry point from outside, routing by domain name and path, TLS.
ConfigMap / Secret	Configuration and secrets injected into pods.
PersistentVolume / PVC	Durable storage for stateful applications.
StatefulSet	For databases: a stable identity and stable storage for each pod.
DaemonSet	One pod on every node (logging and monitoring agents).
Namespace	A logical partition of the cluster (dev, prod, teams).
HorizontalPodAutoscaler	Adds or removes pods based on CPU or other metrics.

9.4 The declarative principle

You do not write "run this"; you describe "I want this" in a YAML file, then run `kubectl apply -f deployment.yaml`. If a pod dies, the controller sees 2 copies instead of 3 and creates a new one: this is **self-healing**. The file is versioned in Git, which leads to **GitOps** (Argo CD, Flux).

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: navmarket-api
spec:
  replicas: 3                                # desired state: 3 pods
  selector:
    matchLabels: { app: navmarket-api }
  template:
    metadata:
      labels: { app: navmarket-api }
    spec:
      containers:
        - name: api
          image: ghcr.io/navatech/navmarket-api:1.0.0
          ports: [{ containerPort: 3000 }]
```

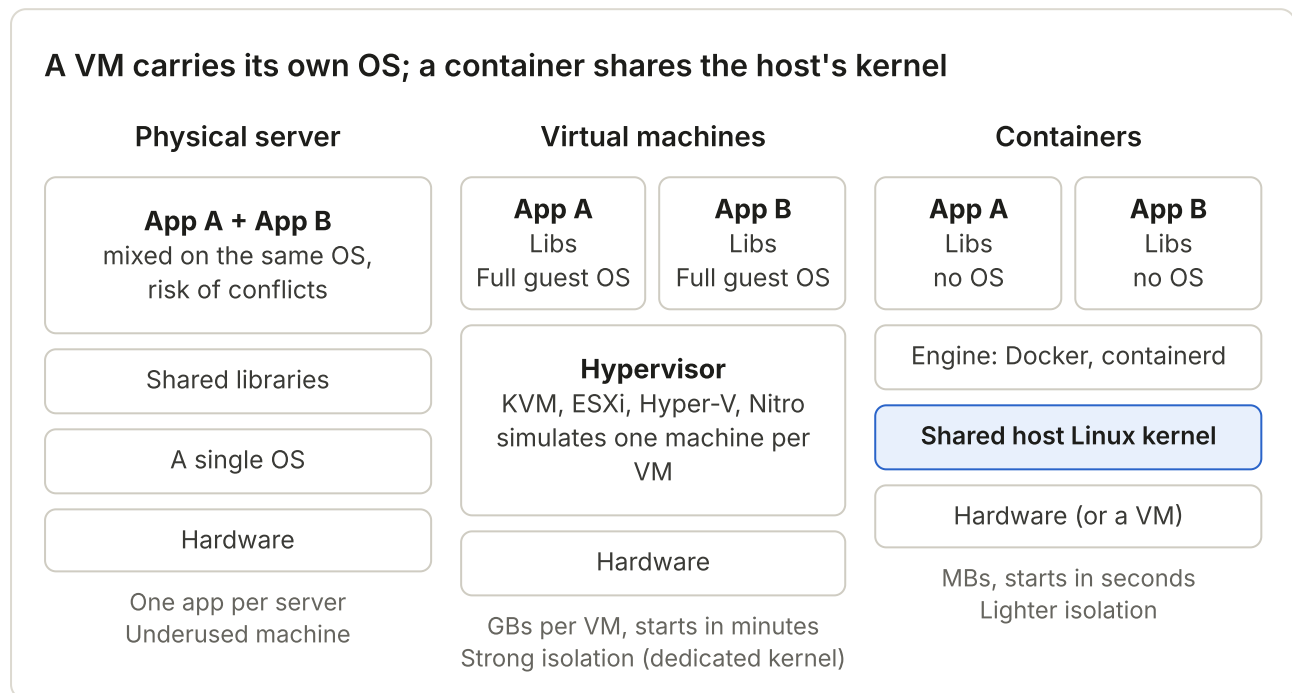
9.5 Where to run Kubernetes

- **Locally, to learn:** kind, minikube, k3s (lightweight, perfect for a homelab).
- **Self-installed:** kubeadm (what you practice for the CKA certification).
- **Managed in the cloud:** Amazon EKS, Azure AKS, Google GKE. The provider runs the control plane; you manage the nodes and your applications.

When not to use Kubernetes: a small application, a small team, little traffic. Docker Compose on a VPS, or a service like ECS Fargate, is often simpler and cheaper. Kubernetes brings power, but also a lot of complexity.

10. Virtualization vs Docker vs Kubernetes vs cloud

These four notions are not competitors: they answer different questions and stack on top of each other. Virtualization splits a machine, Docker packages an application, Kubernetes drives containers across several machines, and the cloud rents all of this to you on demand.



Physical server, VMs and containers: what each one stacks.

The key difference fits in one line: each VM carries a complete OS, whereas containers share the host's kernel and only bring the application and its libraries.

Criterion	Virtual machine	Container (Docker)	Kubernetes	Cloud
Question it answers	How do I run several OSes on one server?	How do I ship an app that runs the same everywhere?	How do I drive hundreds of containers across several machines?	How do I get all of this without buying or managing hardware?
What you handle	A complete machine	An application and its dependencies	A desired state (YAML) for a whole cluster	Services rented through APIs
Level virtualized	The hardware	The operating system	A cluster of machines	An entire datacenter
Size and startup	GBs, minutes	MBs, seconds	Depends on the containers	Seconds to minutes

Criterion	Virtual machine	Container (Docker)	Kubernetes	Cloud
Isolation	Strong (dedicated kernel)	Lighter (shared kernel)	That of the containers, plus network policies	Whatever the provider chose, plus your own (IAM, networks)
Tools	KVM, VMware, Hyper-V, Proxmox, VirtualBox	Docker, Podman, containerd	Kubernetes, k3s, EKS, AKS, GKE	AWS, Azure, Google Cloud, OVHcloud
Good for	Running different OSes, strong isolation, legacy apps	Microservices, CI/CD, a dev environment identical to production	Large production apps, high availability, automatic scaling	Starting fast, absorbing peaks, managed services, global reach
In the NavMarket programme	S7 (KVM, Proxmox), S8 (Vagrant)	S11	S19-21 (CKA)	S13-16 (AWS), S17-18 (Terraform)

How it all stacks up in real life: on Amazon EKS, the NavMarket container runs in a Kubernetes pod, on a node that is an EC2 VM, created by the Nitro hypervisor, on a physical server in an AWS datacenter in the Paris region. All four layers are present at once.

Which one to choose, in short

- A whole OS to test or isolate (Windows, a firewall, a lab you intend to break) → **VM**.
- An app to ship reproducibly → **container**; several containers on one server → **Docker Compose**.
- Many containers, several servers, zero downtime → **Kubernetes** (or ECS for something simpler on AWS).
- No wish to buy or manage hardware, a need for elasticity → **cloud**, whatever technology runs on top.

11. Reliable sources to go further

Always favor the official documentation of each tool. For French-speaking readers, Stéphane Robert's training site, written for DevOps engineers, covers networking, Linux, containers, Kubernetes and AWS in the same order as this programme.

Topic	Resource	Language	Format
Computing basics (hardware, OS)	Crash Course Computer Science, episode 1 (a series of 40 videos of ~12 min)	English	Video
Linux	Linux Journey	English	Free interactive course
Networking for beginners	Les réseaux de zéro (Zeste de Savoir)	French	Complete tutorial
Networking for DevOps	Network fundamentals and OSI and TCP/IP models (Stéphane Robert)	French	Guide
Linux network troubleshooting	Diagnosing a network problem on Linux	French	Practical guide
Networking, short explanations	Cloudflare Learning: the OSI model and What is a protocol?	English	Articles
Networking, structured course	Cisco Networking Academy: Networking Basics	English	Free course with a badge
Virtualization	Red Hat: What is a hypervisor? and What is KVM?	English	Articles
Cloud, official definition	NIST SP 800-145 (PDF, 7 pages)	English	Standard
AWS cloud	What is cloud computing? (AWS) and AWS Cloud Essentials	English	Whitepaper, learning path
Shared responsibility	AWS shared responsibility model	English	Whitepaper
VMs vs containers	Google Cloud: containers vs VMs , AWS: the difference between containers and VMs , Red Hat: containers vs VMs	English	Articles
Docker	What is Docker? (official docs), then the Docker workshop	English	Docs + hands-on workshop
Kubernetes	Kubernetes: Overview , then Learn Kubernetes Basics	English	Docs + interactive tutorial

Topic	Resource	Language	Format
Kubernetes in French	Kubernetes training (Stéphane Robert)	French	Complete guide

Suggested reading order to get started: Crash Course episodes 1 to ~20 in the background (hardware and OS), Linux Journey alongside S1-2, then a networking course (Networking Basics, or Les réseaux de zéro for French readers) before S3. The cloud, Docker and Kubernetes resources can wait for their own session.

12. Self-assessment

You have mastered the basics when you can answer these questions out loud, without notes. Tick them off session by session.

Hardware and OS

- What roles do the CPU, the RAM and the disk play? What happens when the RAM is full?
- Describe how a Linux machine boots, from UEFI to systemd.
- What is a system call, and why doesn't a program talk to the hardware directly?
- What is the difference between a program, a process and a service?

Networking

- Name the TCP/IP layers and one protocol for each layer.
- MAC, IP, port: what is each one for, and which one changes at every router?
- How many hosts are in a /26? Split `10.0.0.0/16` into 4 equal subnets.
- TCP or UDP for DNS, SSH, video calls? Why?
- Explain the three-way handshake and the difference between an RST reply and a timeout.
- Describe what happens when you type a URL into a browser.
- What are ARP, DHCP, DNS and NAT used for?

Virtualization and cloud

- What is the difference between a type 1 and a type 2 hypervisor? Which category does KVM belong to?
- What is the difference between a VM's NAT and bridge network modes?
- What are the 3 ingredients that take you from virtualization to the cloud?
- IaaS, PaaS, SaaS: who manages the OS in each case? Give an example of each.
- Region vs Availability Zone: why deploy across 2 AZs?
- In the shared responsibility model, who patches the OS of an EC2 instance?

Containers and Kubernetes

- Why does a container start in a second and a VM in a minute?
- Image vs container? Why must data go in a volume?
- What are namespaces and cgroups?
- Pod, Deployment, Service: explain the role of each one.
- What does Kubernetes do if a node fails?
- For the NavMarket shop, when would you choose Docker Compose on a VPS, ECS Fargate or EKS?